11. Presentación de Stack Tecnológico

El stack tecnológico es el conjunto de herramientas con las que se planifica, construye, prueba, despliega y opera el Sistema de Control Transmetro. Esta sección presenta el stack del proyecto completo, desde el desarrollo hasta la puesta en producción para la municipalidad, y justifica cada elección con criterios medibles. Se incluyen tanto los componentes que ya funcionan en el prototipo como los planificados para el cierre del proyecto.

La solución es una aplicación web de tres capas: el navegador de la PC de trabajo de cada estación (presentación), una aplicación Laravel alojada en Render (lógica de negocio) y una base de datos PostgreSQL administrada por Supabase (datos y reglas de integridad). Ningún equipo de estación se conecta directamente a la base de datos; toda operación pasa por la aplicación, que valida al usuario, sus permisos y las reglas del enunciado.

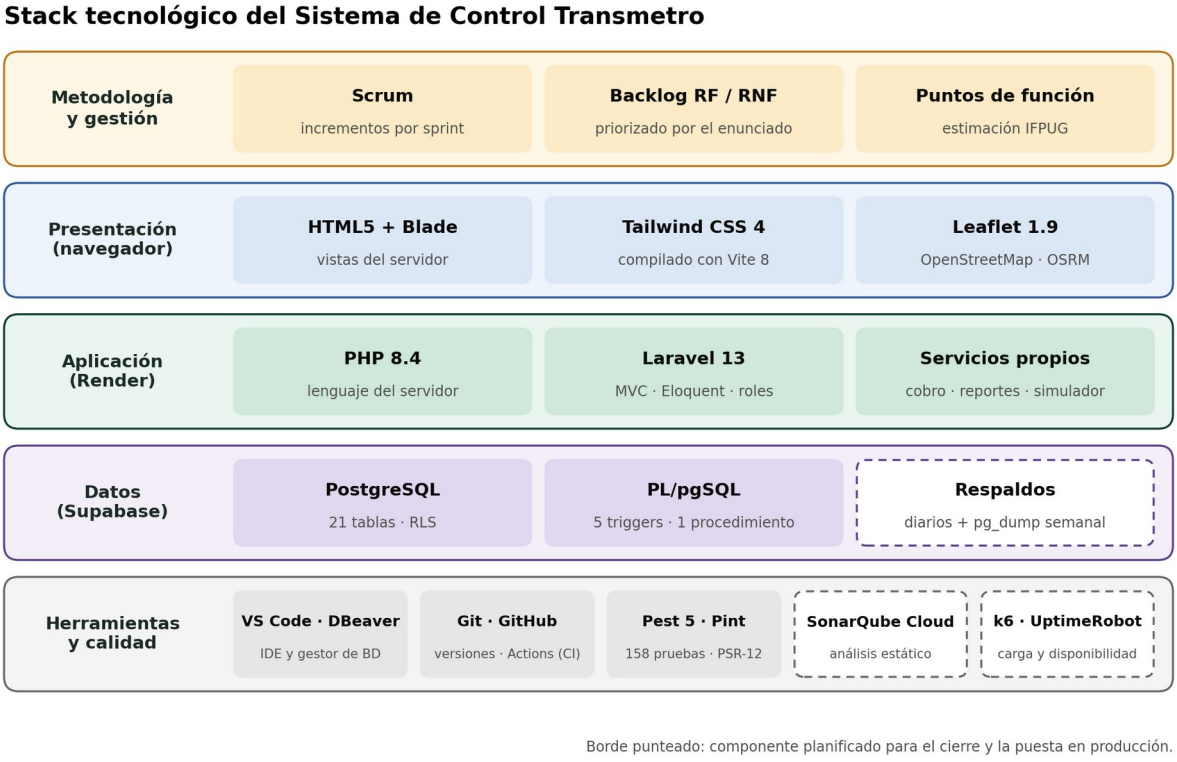


Figura 9. Stack tecnológico del Sistema de Control Transmetro por capas.

11.1. Criterios de selección

Cada componente principal se comparó con dos alternativas usando una matriz de decisión ponderada. Cada alternativa recibe una calificación de 1 (deficiente) a 5 (excelente) por criterio; la calificación final es el promedio ponderado. Los criterios generales fueron:

Tabla 11.1. Criterios generales de selección del stack.

Criterio	Qué se evaluó
Cumplimiento técnico	Que la herramienta permita implementar las reglas del enunciado (integridad, triggers, roles, reportes) sin soluciones improvisadas.
Costo	Licencias y servicios necesarios en desarrollo y en producción.
Seguridad	Mecanismos incluidos: cifrado, control de acceso, protección contra ataques comunes.
Despliegue en la nube	Facilidad para publicar la aplicación (punto 12) y conectar las estaciones (punto 17).
Comunidad y soporte	Documentación, actualizaciones de seguridad y tiempo de soporte de la versión.
Dominio del equipo	Experiencia previa y curva de aprendizaje dentro del plazo del curso.

11.2. Base de datos: PostgreSQL en Supabase

PostgreSQL es un sistema gestor de bases de datos relacional de código abierto (PostgreSQL License), reconocido por su apego al estándar SQL y por su manejo de integridad. En producción se usa **Supabase**, un servicio que administra PostgreSQL en la nube (servidores de AWS en la región este de Estados Unidos) e incluye conexión cifrada, respaldos y monitoreo. En el entorno de pruebas automáticas se usa PostgreSQL 16.

La razón principal de la elección es que las reglas del enunciado se aplican **dentro de la base de datos**: aunque alguien intentara modificar los datos fuera de la aplicación, la base rechaza cualquier operación que viole una regla.

Tabla 11.2. Características de PostgreSQL utilizadas en el proyecto.

Característica	Uso en el Sistema de Control Transmetro
Llaves foráneas (28)	Relaciones entre municipios, líneas, estaciones, accesos, buses, personal, tarjetas y pagos.
Restricciones CHECK (39)	Valores permitidos: recargas de Q5 a Q200, ocupación de 0 a 100 %, estados válidos, métodos de pago sin efectivo.
Triggers en PL/pgSQL (5)	Mínimo 2 accesos por estación, mínimo 1 guardia por acceso, máximo 2n buses por línea, alerta al 150 % del aforo y distancia total de la línea.
Procedimiento almacenado (1)	sp_cambiar_parqueo_bus: cambia el parqueo de un bus en una sola transacción, sin que quede sin parqueo (punto 7).
Transacciones y bloqueo de filas	El cobro del pasaje bloquea la tarjeta (SELECT … FOR UPDATE) para que dos pases simultáneos no gasten el mismo saldo.
UUID y JSONB	Identificadores únicos difíciles de adivinar; el trazo de cada línea se guarda como JSONB.
Row Level Security	Activado en las 21 tablas: bloquea el acceso directo desde la API pública de Supabase.
Índices (65)	Consultas rápidas por línea, estación, bus y fecha en paneles y reportes.

Tabla 11.3. Comparación de gestores de base de datos (calificación de 1 a 5).

Criterio (peso)	PostgreSQL + Supabase	MySQL 8	SQL Server Express
Reglas e integridad en la BD (30 %)	5	4	5
Costo (20 %)	5	5	3
Hospedaje administrado accesible (20 %)	5	4	2
Rendimiento y escalabilidad (15 %)	5	4	3
Dominio del equipo (15 %)	4	4	3
Calificación ponderada	**4.85**	**4.20**	**3.40**

Tabla 11.4. Entornos de la base de datos.

Entorno	Servicio	Características
Desarrollo	Supabase (plan gratuito) y PostgreSQL local	Datos de prueba de las 8 líneas oficiales; se pausa tras 7 días sin uso; sin respaldos automáticos.
Pruebas automáticas	Esquema “pruebas” y PostgreSQL 16 en GitHub Actions	Base aislada que se crea y destruye en cada ejecución; nunca toca los datos reales.
Producción	Supabase Pro (US$25/mes)	Sin pausas por inactividad, respaldo diario con 7 días de retención, conexión SSL por el pooler de sesiones; respaldo semanal adicional con pg_dump.

11.3. Metodología de desarrollo: Scrum

Scrum es un marco de trabajo ágil que organiza el desarrollo en ciclos cortos llamados sprints, al final de los cuales se entrega una parte funcional del sistema. Se eligió porque los requerimientos del Transmetro se fueron precisando durante el proyecto (por ejemplo, la incorporación de las líneas y horarios oficiales y de la Tarjeta Ciudadana) y porque el curso exige revisiones frecuentes del avance.

Tabla 11.5. Roles, artefactos y eventos de Scrum aplicados al proyecto.

Elemento	Aplicación en el proyecto
Product Owner	Representante de la municipalidad: define y prioriza los requerimientos a partir de los puntos del enunciado.
Scrum Master	La consultora: organiza los sprints, elimina impedimentos y cuida que se cumpla la definición de terminado.
Equipo de desarrollo	La consultora: analiza, programa, prueba y documenta.
Product Backlog	Requerimientos funcionales RF-01 a RF-22 y no funcionales RNF-01 a RNF-11, priorizados por punto del enunciado.
Sprint Backlog	Historias seleccionadas para el sprint, con sus pruebas automatizadas.
Incremento	Versión funcional con todas las pruebas aprobadas en GitHub Actions.
Eventos	Planificación al inicio del sprint, seguimiento diario, revisión con demostración del sistema y retrospectiva.
Definición de terminado	Código con formato Pint, pruebas nuevas y existentes aprobadas, requerimiento trazado en la matriz y documentación actualizada.

Tabla 11.6. Planificación de sprints del proyecto completo.

Sprint	Fechas	Objetivo	Estado
1	21 – 27 sep	Análisis de requerimientos, modelo entidad relación y script de la base de datos	Terminado
2	28 sep – 4 oct	Migraciones y reglas en la BD, autenticación por roles, módulos de administración, panel del operador, Tarjeta Ciudadana, líneas oficiales con mapa, reportes y simulador	Terminado
3	5 – 11 oct	Bitácora en pantalla, reportes de alertas, recaudación y personal, registro de viajes, SonarQube Cloud y cobertura de pruebas	Planificado
4	12 – 18 oct	Despliegue en Render y Supabase Pro, pruebas de carga y de seguridad, monitoreo de disponibilidad	Planificado
5	19 – 25 oct	Plan de pruebas, criterios de aceptación, manual de usuario y capacitación	Planificado
6	26 – 30 oct	Cierre: documento final, presentación ejecutiva, video, carta de aceptación y entrega	Planificado

Tabla 11.7. Comparación de metodologías (calificación de 1 a 5).

Criterio (peso)	Scrum	Kanban	Cascada
Adaptación a cambios de requerimientos (30 %)	5	5	2
Entregas frecuentes para revisión (25 %)	5	4	2
Documentación exigida por el curso (25 %)	4	3	5
Adecuación al tamaño del equipo (20 %)	4	5	5
Calificación ponderada	**4.55**	**4.25**	**3.35**

11.4. Lenguajes de programación

El lenguaje principal es **PHP 8.4**, versión con soporte activo hasta el 31 de diciembre de 2026 y de seguridad hasta el 31 de diciembre de 2028. Se complementa con SQL para las reglas de negocio en la base de datos y con JavaScript para el mapa y la actualización de pantallas.

Tabla 11.8. Lenguajes utilizados.

Lenguaje	Versión	Dónde se usa	Aporte al proyecto
PHP	8.4	Controladores, modelos, servicios, comandos y pruebas	Tipado estricto, enumeraciones y un ecosistema maduro para aplicaciones web.
SQL y PL/pgSQL	PostgreSQL 15+	Migraciones, triggers, procedimiento y consultas de reportes	Las reglas del enunciado quedan protegidas en la propia base de datos.
JavaScript	ES2022	Mapa de rutas, editor de trazo, simulación y recarga del panel	Interacción en el navegador sin instalar nada en la PC de la estación.
HTML5 y Blade	—	Vistas de la aplicación	Plantillas reutilizables con protección automática contra inyección de código (XSS).
CSS (Tailwind)	Tailwind 4.3	Estilos de todas las pantallas	Interfaz consistente y adaptable a distintos tamaños de pantalla.

11.5. Framework: Laravel 13

Laravel es el framework de PHP más utilizado para aplicaciones web. El proyecto usa la versión **13.33**; Laravel 13 se publicó el 17 de marzo de 2026, recibe correcciones hasta el tercer trimestre de 2027 y actualizaciones de seguridad hasta el 17 de marzo de 2028, y funciona con PHP 8.3 a 8.5. Sigue el patrón **Modelo–Vista–Controlador (MVC)**; en el proyecto se agregó una capa de servicios para la lógica de negocio que no pertenece a un solo modelo.

Tabla 11.9. Componentes de Laravel utilizados.

Componente	Uso en el Sistema de Control Transmetro
Rutas y middleware	Cada ruta exige sesión; el middleware VerificarRol separa las funciones del administrador y del operador.
Eloquent ORM	20 modelos con sus relaciones (línea–estaciones, estación–accesos, bus–parqueo, etc.).
Migraciones (23)	Estructura de la base de datos versionada en el repositorio, incluidos triggers y procedimiento.
Seeders	Carga de las 8 líneas oficiales, 121 estaciones, 173 buses, personal, tarjetas y viajes de prueba.
Validación	Reglas en cada formulario con mensajes en español.
Blade y componentes	Vistas y componentes reutilizables (campos, selectores, títulos).
Seguridad	Contraseñas con bcrypt, protección CSRF y cifrado AES-256 del contacto y la dirección del personal y del historial educativo de los pilotos.
Caché y bloqueos	Estado y eventos del simulador; un bloqueo evita que dos ciclos corran a la vez.
Artisan	Comandos propios: transmetro:simular, transmetro:consultas-reportes y transmetro:guardar-trazos.
Pruebas	Integración con Pest para probar pantallas, reglas y servicios.

Tabla 11.10. Organización del código (capas de la aplicación).

Capa	Contenido
Controladores	Administración (municipios, líneas, estaciones, buses, personal, tarjetas, reportes, simulación), panel del operador, mapa y autenticación.
Servicios	CobroPasaje (pagos), Reportes (consultas SQL), ImportadorKml y ExportadorTrazos (mapas oficiales) y SimuladorOperacion.
Modelos	Municipio, Linea, HorarioLinea, Estacion, Acceso, Molinete, Parqueo, Bus, Empleado, Piloto, Seguridad, Operador, Tarjeta, Recarga, TransaccionPago, AlertaSistema, Viaje, VisitaViaje, BitacoraAuditoria y User.
Vistas	34 plantillas Blade con Tailwind CSS.
Base de datos	Migraciones, triggers, procedimiento almacenado y Row Level Security.

En la capa de presentación se usan **Tailwind CSS 4.3** compilado con **Vite 8.3**, y **Leaflet 1.9.4** con mapas de **OpenStreetMap** para mostrar las líneas y estaciones. El trazo de las rutas se ajusta a las calles con el servicio **OSRM**.

Tabla 11.11. Comparación de frameworks (calificación de 1 a 5).

Criterio (peso)	Laravel (PHP)	Django (Python)	Spring Boot (Java)
Productividad: ORM, migraciones, plantillas (25 %)	5	5	3
Seguridad incluida (20 %)	5	5	5
Dominio del equipo y curva de aprendizaje (20 %)	5	3	2
Hospedaje económico (15 %)	4	4	3
Comunidad y soporte de la versión (20 %)	5	5	5
Calificación ponderada	**4.85**	**4.45**	**3.60**

11.6. IDE y herramientas de desarrollo

El entorno de desarrollo integrado es **Visual Studio Code**, gratuito y liviano, con extensiones para PHP, Laravel, Tailwind y PostgreSQL y una terminal integrada desde la que se ejecutan Composer, Artisan, npm y Git. La base de datos se diseña y consulta con **DBeaver**.

Tabla 11.12. Herramientas del entorno de desarrollo.

Herramienta	Versión	Uso
Visual Studio Code	Última estable	Edición de código, terminal, depuración y control de Git
Extensiones de VS Code	—	PHP Intelephense, Laravel, Tailwind CSS IntelliSense, SQLTools/PostgreSQL y GitLens
DBeaver	26.2	Diagrama entidad relación, scripts SQL y revisión de datos
Composer	2.x	Dependencias de PHP
Node.js y npm	22 LTS	Compilación de estilos con Vite
Git y GitHub	—	Control de versiones y repositorio del proyecto
Pest	5.2	Pruebas automatizadas (unitarias y de funcionalidad)
Laravel Pint	1.32	Formato de código según el estándar PSR-12
PlantUML y Graphviz	1.2025 / 2.43	Diagramas UML y modelo entidad relación

Tabla 11.13. Comparación de entornos de desarrollo (calificación de 1 a 5).

Criterio (peso)	VS Code	PhpStorm	NetBeans
Costo (30 %)	5	3	5
Soporte para PHP y Laravel (30 %)	4	5	3
Rendimiento en la computadora del equipo (20 %)	5	3	3
Integración con Git y terminal (20 %)	5	5	4
Calificación ponderada	**4.70**	**4.00**	**3.80**

11.7. Infraestructura, integración continua y despliegue

El código pasa por un flujo automático antes de llegar a producción: cada cambio que se sube a GitHub ejecuta GitHub Actions, que revisa el formato y corre todas las pruebas sobre una base PostgreSQL limpia. Solo si todo pasa, la nueva versión se publica en Render. SonarQube Cloud se agregará en el sprint 3 para el análisis estático de calidad.

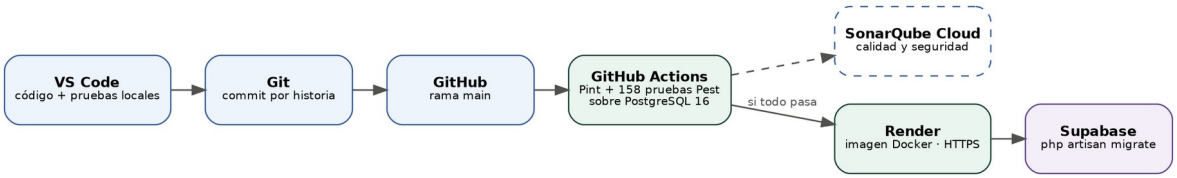


Figura 10. Flujo de integración y despliegue continuo (el borde punteado indica un paso planificado).

Tabla 11.14. Ambientes del proyecto.

Ambiente	Infraestructura	Propósito
Desarrollo	Laptop con Windows: PHP 8.4, Composer, Node 22, VS Code y DBeaver; aplicación en http://localhost:8000	Programar y probar cada historia
Integración continua	GitHub Actions (Ubuntu) con PostgreSQL 16	Verificar formato y 158 pruebas en cada cambio
Producción	Render Starter (contenedor Docker, 512 MB, siempre activo, HTTPS) + Supabase Pro	Servicio para la municipalidad, accesible desde cada estación por Internet

Tabla 11.15. Servicios externos.

Servicio	Uso	Condición
OpenStreetMap	Mapa base de líneas y estaciones	Licencia ODbL: se muestra el crédito “© OpenStreetMap” en el mapa
OSRM	Ajuste del trazo de las rutas a las calles	Solo lo usa el administrador al editar un trazo; el trazo queda guardado en la base
Mapas KML/KMZ oficiales	Importación del recorrido de cada línea	Archivos publicados por la Municipalidad de Guatemala

11.8. Seguridad incorporada en el stack

Tabla 11.16. Controles de seguridad que aporta cada componente.

Riesgo	Control	Componente
Acceso no autorizado	Inicio de sesión, middleware de roles y sesiones en la base de datos	Laravel
Robo de contraseñas	Contraseñas guardadas con bcrypt	Laravel
Falsificación de solicitudes (CSRF)	Token CSRF en todos los formularios	Laravel
Inyección SQL	Consultas preparadas en Eloquent y en los reportes	Laravel / PostgreSQL
Inyección de código (XSS)	Escape automático en las vistas Blade	Blade
Exposición de datos personales	Cifrado AES-256 de contacto, dirección e historial educativo	Laravel
Acceso directo a la base de datos	Row Level Security en las 21 tablas y conexión SSL	Supabase
Intercepción de datos en la red	HTTPS con TLS entre las estaciones y la aplicación	Render
Pérdida de información	Respaldo diario y respaldo semanal con pg_dump	Supabase
Credenciales en el código	Claves solo en el archivo .env, fuera del repositorio	Laravel / Git

11.9. Resumen de versiones, licencias y costos

Tabla 11.17. Resumen del stack tecnológico.

Categoría	Componente	Versión	Licencia	Costo
Base de datos	PostgreSQL / Supabase Pro	15+	PostgreSQL License / servicio	US$25 al mes
Metodología	Scrum	—	—	Q0.00
Lenguaje	PHP	8.4	PHP License	Q0.00
Framework	Laravel	13.33	MIT	Q0.00
IDE	Visual Studio Code	Estable	MIT (código base)	Q0.00
Gestor de BD	DBeaver Community	26.2	Apache 2.0	Q0.00
Interfaz	Tailwind CSS · Vite · Leaflet	4.3 · 8.3 · 1.9.4	MIT · MIT · BSD-2	Q0.00
Pruebas y calidad	Pest · Pint · SonarQube Cloud	5.2 · 1.32 · —	MIT · MIT · plan gratuito	Q0.00
Integración continua	GitHub y GitHub Actions	—	Servicio	Q0.00
Alojamiento	Render Starter	—	Servicio	US$7 al mes

Todo el software del proyecto es de código abierto o gratuito; el único costo recurrente es la operación en la nube en producción (US$32 al mes, unos Q244 al tipo de cambio de referencia), detallado en la propuesta monetaria.

12. Métricas de Software

Las métricas de software permiten medir con datos objetivos el tamaño, la complejidad, la calidad, el avance y el desempeño del sistema, y comparar lo estimado con lo real. Esta sección cubre el proyecto completo en tres momentos: la **estimación** (inicio), el **desarrollo** (valores medidos en el repositorio al 2 de octubre de 2026) y el **cierre y la operación** (metas que se verificarán antes de la entrega y en producción).

Los valores de código se obtuvieron con un analizador de tokens de PHP aplicado al repositorio, los de pruebas con el reporte JUnit de Pest y los de base de datos con consultas al catálogo de PostgreSQL. Las métricas pendientes indican la herramienta con la que se obtendrán.

12.1. Marco de medición

Las métricas se definieron con el enfoque **GQM (Objetivo – Pregunta – Métrica)**, que parte de lo que el proyecto necesita saber, y se relacionan con las características de calidad del modelo **ISO/IEC 25010**.

Tabla 12.1. Objetivos, preguntas y métricas (GQM).

Objetivo	Pregunta	Métricas
Cumplir el alcance del enunciado	¿Qué parte del sistema está construida?	Puntos de función implementados, requerimientos completos, trazabilidad
Asegurar que las reglas funcionan	¿Se cumplen los puntos del enunciado sin errores?	Pruebas aprobadas, reglas con prueba, defectos y densidad de defectos
Mantener el código fácil de cambiar	¿El código es simple y ordenado?	Complejidad ciclomática, métodos por clase, duplicación, deuda técnica
Ofrecer un servicio confiable	¿El sistema responde rápido y está disponible?	Tiempo de respuesta, disponibilidad, tasa de errores, tiempo de recuperación
Controlar costo y plazo	¿El avance corresponde a lo estimado?	Esfuerzo estimado y restante, productividad, avance por sprint

Tabla 12.2. Relación con las características de calidad de ISO/IEC 25010.

Característica	Cómo se mide en el proyecto
Adecuación funcional	Requerimientos completos, funciones implementadas y reglas del enunciado cubiertas por pruebas
Eficiencia de desempeño	Tiempo de respuesta (percentil 95) con usuarios simultáneos
Compatibilidad	Funcionamiento en Chrome, Edge y Firefox
Usabilidad	Clics por operación frecuente y cuestionario SUS a operadores
Fiabilidad	Disponibilidad, defectos abiertos y tiempo medio de recuperación
Seguridad	Rutas protegidas por rol, vulnerabilidades detectadas, tablas con RLS
Mantenibilidad	Complejidad ciclomática, duplicación, deuda técnica y cobertura de pruebas
Portabilidad	Despliegue reproducible con Docker y migraciones

12.2. Métricas de estimación del proyecto completo

El tamaño funcional se estimó con **Puntos de Función (IFPUG)**, como se detalla en la sección 7. Estos valores son la línea base contra la que se mide el avance.

Tabla 12.3. Línea base de estimación.

Métrica	Valor
Funciones (transacciones EI, EO y EQ)	41 (21 EI, 11 EO, 9 EQ)
Archivos lógicos internos / de interfaz externa	14 ILF / 4 EIF
Puntos de función sin ajustar (PFSA)	343
Factor de ajuste (suma de 14 factores = 57)	1.22
Puntos de función ajustados (PFA)	418.46
Esfuerzo estimado	2,510.76 horas hombre (6 horas por PF)
Duración estimada	7.47 meses con 2 desarrolladores
Costo de desarrollo estimado	Q99,010.63
Líneas de código estimadas	20,923 (50 líneas por PF, lenguaje de 4.ª generación)

12.3. Métricas de tamaño

Líneas de código por capa

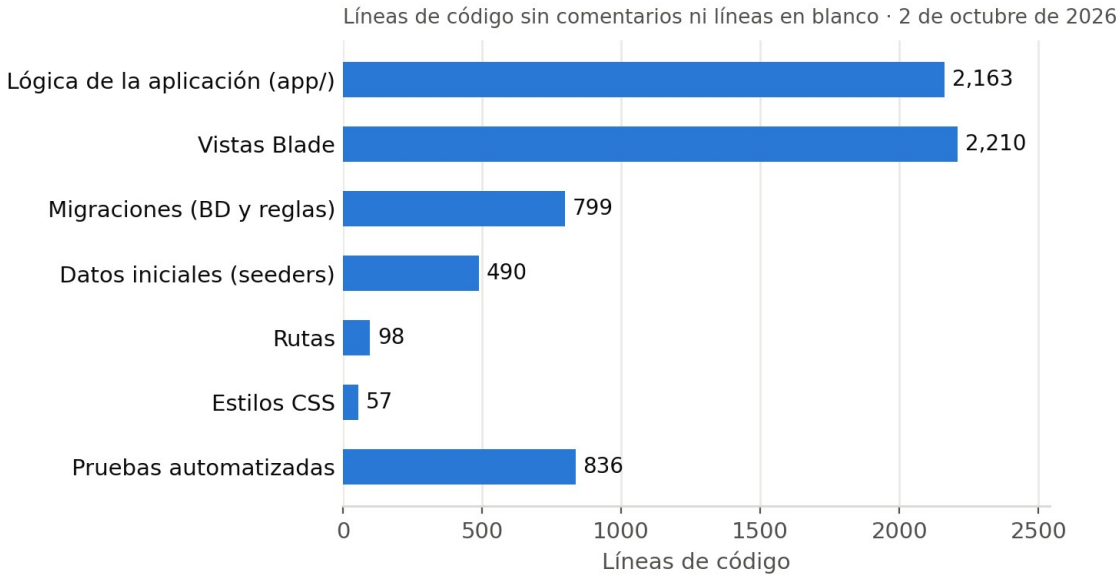


Figura 11. Líneas de código por capa del sistema.

Tabla 12.4. Tamaño del código al 2 de octubre de 2026.

Capa	Archivos	Líneas físicas	Líneas de código	Comentarios
Lógica de la aplicación (app/)	43	3,905	2,163	344
Vistas Blade	34	2,210	2,210	—
Migraciones (estructura y reglas)	23	1,233	799	132
Datos iniciales (seeders y mapas)	5	755	490	89
Rutas y comandos	2	143	98	10
Estilos CSS	1	57	57	—
Total de producción	**108**	**8,303**	**5,817**	**575**
Pruebas automatizadas (tests/)	16	1,387	836	96

Indicadores derivados: 43 clases y 212 métodos en la lógica de la aplicación (4.9 métodos por clase), densidad de comentarios del 13.7 % en app/, 20 modelos, 14 controladores y 5

servicios; además, el script SQL de 565 líneas y el archivo de consultas de reportes de 123 líneas.

Tabla 12.5. Tamaño proyectado al cierre del proyecto.

Métrica	Actual	Proyectado al cierre	Base de la proyección
Puntos de función ajustados	390.40	418.46	Alcance de la sección 7
Líneas de código de producción	5,817	≈ 6,240	28.06 PF pendientes × 14.9 líneas por PF observadas
Pruebas automatizadas	158	≥ 190	Al menos 4 pruebas por función pendiente y pruebas de seguridad
Esfuerzo restante	—	≈ 168 horas hombre	28.06 PF × 6 horas por PF

12.4. Métricas de complejidad y mantenibilidad

La **complejidad ciclomática de McCabe** cuenta los caminos independientes de un método (1 + número de decisiones: if, for, foreach, while, case, catch, &&, ||, ?? y ?). Un valor de 10 o menos se considera fácil de probar y mantener.

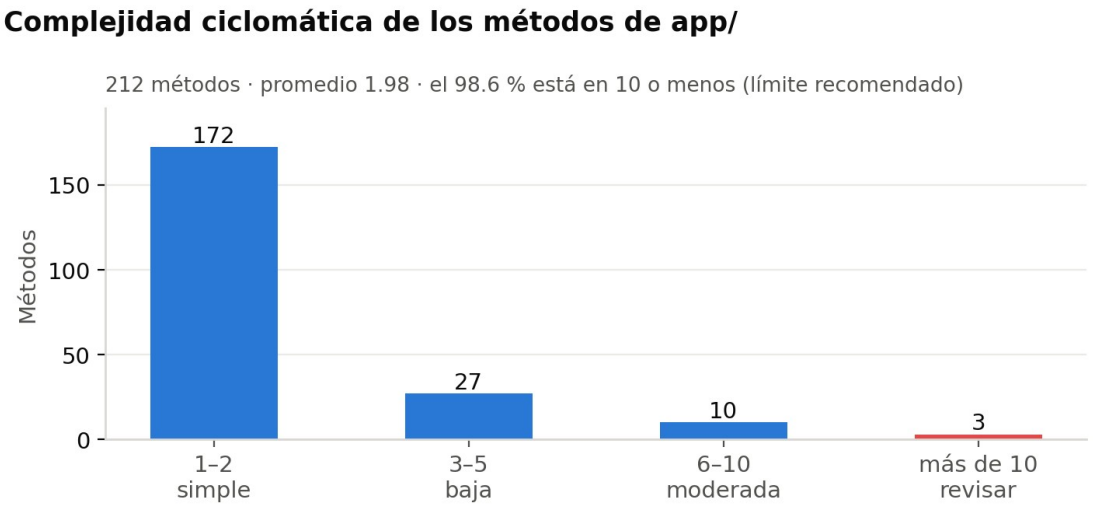


Figura 12. Distribución de la complejidad ciclomática de los métodos de la aplicación.

Tabla 12.6. Complejidad ciclomática por capa.

Indicador	app/	Migraciones	Seeders	Pruebas	Meta
Promedio por método	1.98	1.04	7.79	1.11	≤ 5
Máxima	18	2	60	3	≤ 15
Métodos con más de 10	3	0	2	0	0 en app/

Métodos que se refactorizarán en el sprint 3 por superar el límite: ImportadorKml::encadenar (18), ImportadorKml::leer (14) y TrazoLineaController::puntos (13). En los datos iniciales, TransmetroDatos::generar (60) y TransmetroDatos::completarLinea (11) concentran la carga de las líneas oficiales; no forman parte de la operación diaria, pero se dividirán en métodos más pequeños.

Tabla 12.7. Métricas de mantenibilidad que se obtendrán con SonarQube Cloud.

Métrica	Meta	Estado
Código duplicado	Menos de 3 %	Pendiente (sprint 3)
Deuda técnica	Calificación A (menos de 5 % del esfuerzo de desarrollo)	Pendiente (sprint 3)
Code smells críticos	0	Pendiente (sprint 3)
Complejidad cognitiva por método	≤ 15	Pendiente (sprint 3)

12.5. Métricas de la base de datos

Tabla 12.8. Métricas del modelo de datos.

Indicador	Valor
Tablas del dominio + tabla de usuarios	20 + 1
Columnas	135
Llaves foráneas	28
Restricciones CHECK	39
Índices	65
Triggers / procedimientos almacenados	5 / 1
Tablas con Row Level Security	21 de 21 (100 %)
Reglas de integridad del enunciado aplicadas en la base	7 (puntos 3, 5, 7, 10, 13, 14 y 18)
Datos de prueba cargados	8 líneas, 121 estaciones, 173 buses, 254 accesos, 189 empleados

12.6. Métricas de calidad: pruebas

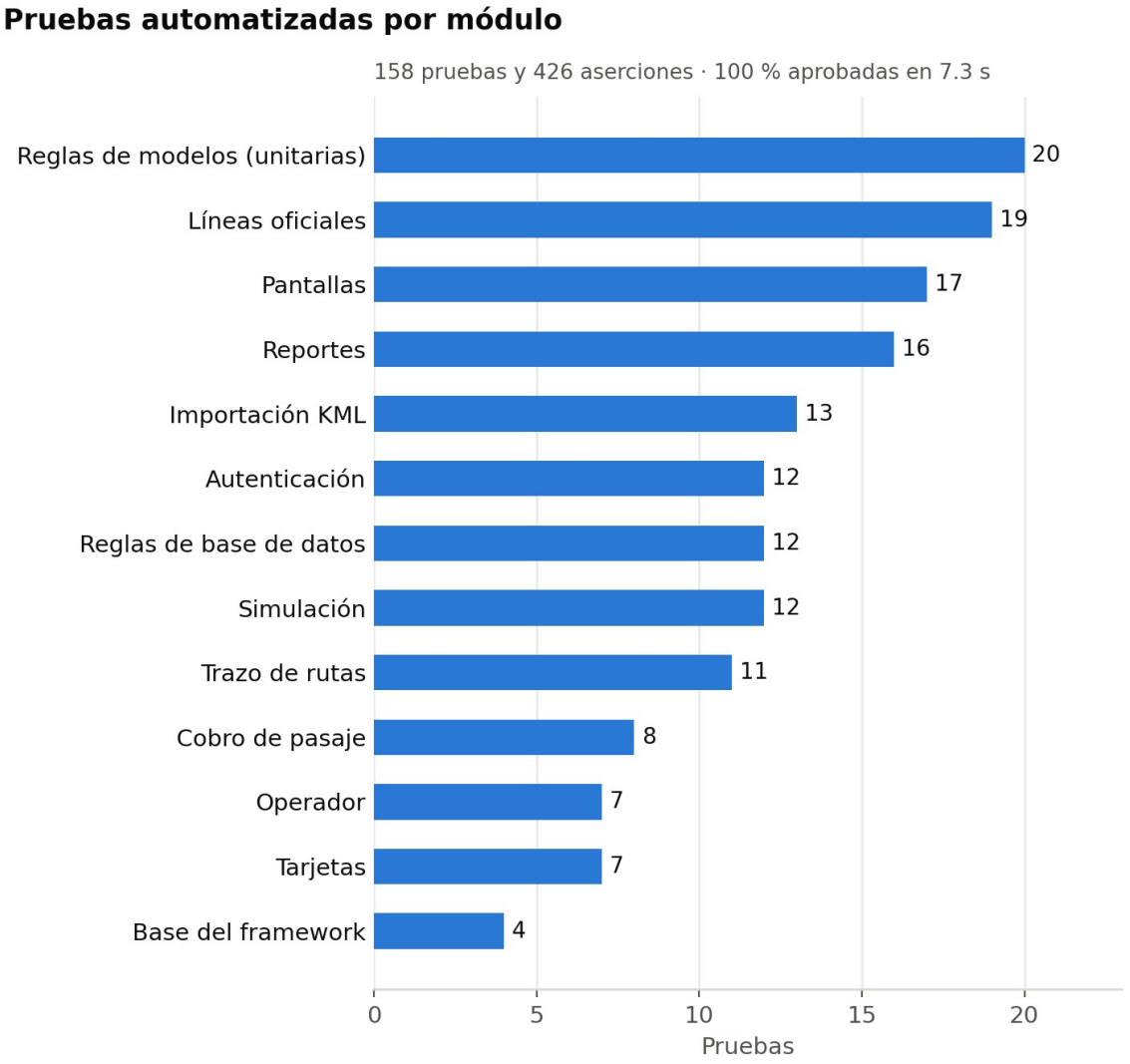


Figura 13. Pruebas automatizadas por módulo.

Tabla 12.9. Indicadores de pruebas.

Indicador	Valor actual	Meta al cierre
Pruebas automatizadas	158 (21 unitarias y 137 de funcionalidad)	≥ 190
Aserciones	426 (2.7 por prueba)	≥ 2 por prueba
Pruebas aprobadas	100 %	100 %
Tiempo de ejecución	7.3 segundos	Menos de 60 segundos
Reglas del enunciado con al menos una prueba	9 de 9 (puntos 3, 5, 7, 10, 13, 14, 15, 16 y 18)	100 %
Relación líneas de prueba / líneas de app/	0.39	≥ 0.40
Cobertura de código	Pendiente (requiere PCOV en GitHub Actions)	≥ 80 % en servicios y modelos
Ejecución automática	En cada cambio subido a GitHub	Igual

12.7. Métricas de calidad: defectos

Tabla 12.10. Defectos encontrados y corregidos durante el desarrollo.

No.	Defecto	Detectado en	Estado
1	El procedimiento sp_cambiar_parqueo_bus fallaba al cambiar el parqueo de un bus	Pruebas automatizadas	Corregido
2	El trigger de seguridad no impedía eliminar al último guardia de un acceso	Pruebas automatizadas	Corregido
3	Fechas y horas guardadas en UTC en lugar de la hora de Guatemala	Revisión funcional	Corregido
4	Identificador HTML duplicado en el selector de rol	Revisión funcional	Corregido
5	Las tarjetas de prueba no recibían los 5 viajes gratis	Pruebas automatizadas	Corregido
6	Texto ilegible sobre los colores claros de algunas líneas	Revisión del usuario	Corregido
7	Líneas ordenadas alfabéticamente (L1, L12, L13, L18, L2) en lugar de en orden natural	Revisión del usuario	Corregido
8	El trazo de las rutas no seguía las calles	Revisión del usuario	Corregido
9	Las pruebas podían borrar los datos de desarrollo	Revisión técnica	Corregido

Tabla 12.11. Indicadores de defectos.

Indicador	Valor actual	Meta	Cálculo
Densidad de defectos	1.55 por KLOC	≤ 2 por KLOC	9 defectos ÷ 5.817 KLOC
Defectos cerrados	100 %	100 % antes de la entrega	9 de 9
Eficiencia de remoción (DRE)	100 % hasta hoy	≥ 95 %	Defectos antes de la entrega ÷ defectos totales
Detectados por pruebas automatizadas	33 %	≥ 50 %	3 de 9
Defectos críticos abiertos	0	0	—

12.8. Métricas de avance y de proceso

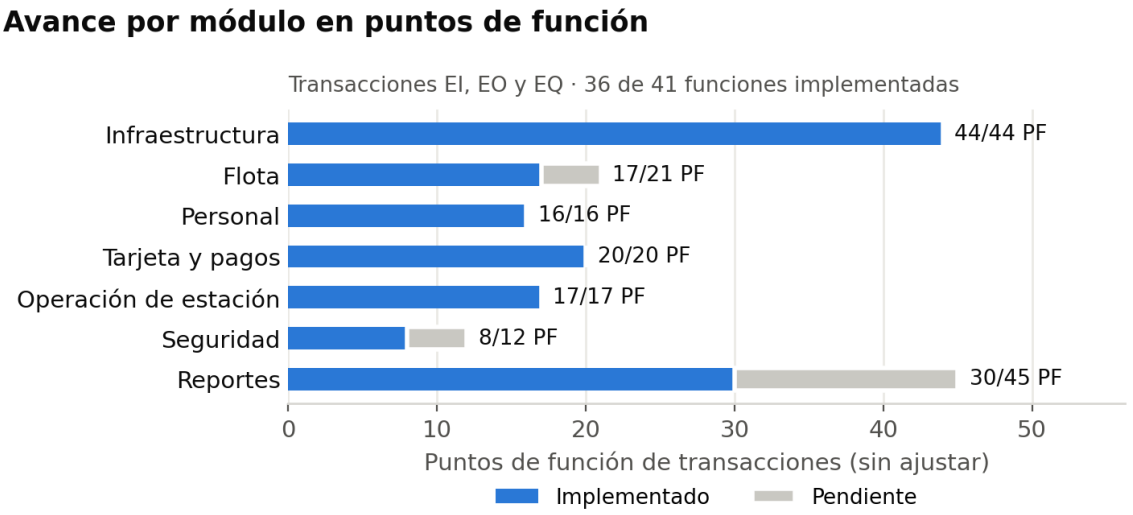


Figura 14. Avance por módulo en puntos de función.

Tabla 12.12. Avance del proyecto.

Indicador	Valor	Cálculo
Avance en puntos de función	93.3 %	390.40 ÷ 418.46 PFA
Funciones implementadas	87.8 %	36 de 41
Requerimientos funcionales completos	72.7 %	16 completos y 6 parciales de 22
Sprints terminados	2 de 6	Tabla 11.6
Productividad observada	14.9 líneas por PF	5,817 ÷ 390.40
Esfuerzo restante estimado	≈ 168 horas hombre	28.06 PF × 6 horas

La productividad real (14.9 líneas por punto de función) es mucho menor que la usada en la estimación (50), porque Laravel resuelve gran parte del trabajo: autenticación, validación, acceso a datos y migraciones. Para estimar futuros proyectos con este stack conviene calibrar el valor en unas 15 líneas por punto de función.

12.9. Métricas de desempeño y operación

Estas métricas se medirán en el sprint 4, con el sistema publicado en Render y Supabase, y luego durante la operación.

Tabla 12.13. Métricas de operación.

Métrica	Meta	Herramienta
Tiempo de respuesta (percentil 95) de consultas, pases y aforo	Menos de 2 segundos con 50 usuarios simultáneos	k6
Tasa de errores del servidor (5xx)	Menos de 1 % de las solicitudes	Registros de Render
Disponibilidad en horario de servicio (04:30 a 21:00)	99 % o más	UptimeRobot
Tiempo de generación de la alerta de refuerzo	En la misma transacción que actualiza el aforo (menos de 1 segundo)	Pruebas y bitácora
Tiempo medio de recuperación (MTTR)	Menos de 48 horas	Issues de GitHub
Satisfacción de los operadores (SUS)	70 puntos o más	Cuestionario SUS

12.10. Tablero resumen de indicadores

Tabla 12.14. Estado de los indicadores clave al 2 de octubre de 2026.

Indicador	Meta	Valor actual	Estado
Pruebas aprobadas	100 %	100 %	Cumple
Reglas del enunciado probadas	100 %	100 %	Cumple
Complejidad promedio en app/	≤ 5	1.98	Cumple
Métodos de app/ con complejidad mayor a 10	0	3	En corrección
Densidad de defectos	≤ 2 por KLOC	1.55	Cumple
Tablas con Row Level Security	100 %	100 %	Cumple
Avance en puntos de función	100 %	93.3 %	En progreso
Requerimientos completos	100 %	72.7 %	En progreso
Cobertura de código	≥ 80 %	Sin medir	Pendiente
Calificación de SonarQube Cloud	A	Sin medir	Pendiente
Tiempo de respuesta p95	< 2 s	Sin medir	Pendiente
Disponibilidad	≥ 99 %	Sin medir	Pendiente

12.11. Plan de medición

Tabla 12.15. Cuándo y con qué se mide cada grupo de métricas.

Grupo de métricas	Herramienta	Frecuencia	Responsable
Pruebas y formato	Pest y Pint en GitHub Actions	En cada cambio	Equipo de desarrollo
Tamaño y complejidad	Analizador de código y SonarQube Cloud	Al cerrar cada sprint	Scrum Master
Cobertura	Pest con PCOV	Al cerrar cada sprint	Equipo de desarrollo
Defectos	Issues de GitHub	Continua	Equipo de desarrollo
Avance	Matriz de trazabilidad y puntos de función	En cada revisión de sprint	Product Owner y Scrum Master
Desempeño	k6	Antes de la entrega y en cada versión mayor	Equipo de desarrollo
Disponibilidad	UptimeRobot	Continua en producción	Municipalidad / soporte

12.12. Conclusiones de las métricas

- El proyecto lleva el **93.3 % de los puntos de función** estimados; lo pendiente se concentra en reportes (alertas, recaudación y personal), la bitácora en pantalla y el registro de viajes.
- La calidad medida es alta: **158 pruebas aprobadas**, todas las reglas del enunciado con prueba automática y una densidad de **1.55 defectos por KLOC**, todos corregidos.
- El código es mantenible (complejidad promedio de 1.98); tres métodos de la importación de mapas se refactorizarán para bajar de 10.
- Quedan por medir la cobertura, el análisis de SonarQube Cloud, el desempeño y la disponibilidad, planificados para los sprints 3 y 4.